ACADEMIC PROJECT REPORT

Electricity Consumption Forecasting at Scale

A Cluster-Level Machine Learning Framework for Industrial Demand Forecasting Using the UCI LD2011–2014 Dataset

GitHub: https://github.com/bhuvighosh3/Forecasting-project

Submitted by

Afreen Sorathiya · Anamika Kumari Mishra · Bhuvi Ghosh

Dataset: UCI LD2011–2014 | 370 Industrial Clients | 15-Minute Resolution | 3 Years

1. Problem Statement and Business Value

1.1 Problem Statement

The global electricity grid increasingly relies on accurate, granular demand forecasts to maintain supply-demand balance, schedule procurement contracts, and optimise grid infrastructure investment. The UCI LD2011–2014 dataset captures real electricity consumption records from 370 industrial and commercial clients at 15-minute resolution over three years (January 2011 – December 2014), yielding approximately 52 million individual meter readings.

The central challenge this project addresses is fourfold:

- Scale: 370 clients each generating 96 readings per day produces over 140,000 data points per client per year. Manual, per-client forecasting by energy analysts is operationally infeasible at this scale.
- Heterogeneity: Clients range from single industrial facilities averaging 23,778 kWh per 15-minute slot to large aggregates averaging as little as 59 kWh. No single model architecture can serve all consumption profiles with acceptable accuracy.
- Data Quality: Real-world meter data contains missing timestamps, forward-fill imputation requirements, IQR and Z-score outlier contamination, and stationarity conditions that must be validated before any forecasting model is applied.
- Forecast Horizon: Grid operators require reliable 30-day-ahead demand forecasts to schedule energy procurement, plan maintenance windows, price Time-of-Use contracts, and manage peak-demand surcharges.

1.2 Business Value Delivered

The cluster-level forecasting framework developed in this project delivers measurable value across four business dimensions:

1.2.1 Operational Cost Reduction

Accurate 30-day demand forecasts enable grid operators and industrial energy managers to purchase electricity at base-rate forward contracts rather than expensive spot-market peak-rate transactions. A 1% improvement in MAPE on a 40-client cluster with mean consumption of 749 kWh per 15-minute slot translates directly into reduced over-procurement costs and avoided peak-demand penalties. The LSTM model deployed on Cluster 4 achieves MAPE 1.74% — a 5× improvement over the industry-baseline Prophet model — representing a substantial reduction in forecast-driven procurement waste.

1.2.2 Risk Mitigation via Probabilistic Forecasting

The framework provides 95% prediction intervals on all deployed forecasts: Prophet's built-in confidence intervals, SARIMA's conf_int() output, and Monte Carlo dropout for LSTM. Probabilistic forecasts replace

deterministic point estimates that systematically under-procure during demand spikes, enabling risk-adjusted procurement decisions and reducing unplanned grid imbalance events.

1.2.3 Model Governance Scalability

The cluster-level pipeline reduces model maintenance overhead from 370 individual client models to 6 cluster models — a 98% reduction in model configurations to monitor, retrain, and validate. This makes the forecasting system maintainable by a small data science team without per-client specialist knowledge, and enables standardised MLOps governance across the entire client portfolio.

1.2.4 Contract Pricing Accuracy

Cluster-calibrated demand forecasts enable precise Time-of-Use and demand-charge tariff modelling. Utility pricing teams can use cluster-level load profiles to design differentiated contracts for client segments, improving revenue forecasting accuracy and enabling more competitive tariff structures for high-value industrial clients.

Key Business Metrics

Best MAPE Achieved: 1.74% (LSTM on Cluster 4 — best-in-class for 15-min industrial energy forecasting)

MAE Reduction: 42% lower MAE with SARIMA vs Prophet on single industrial clients (C1: 645 vs 1,106 kWh)

Governance Savings: 98% reduction in model maintenance (370 individual models → 6 cluster models)

Production-Ready Clusters: 4 of 6 clusters (C0, C1, C3, C4) cleared for deployment

2. Process and Change Management

2.1 Current State (AS-IS)

Prior to this project, the electricity forecasting process operated as follows:

- Forecasting Method: Manual, per-client statistical forecasting performed by energy analysts using a single homogeneous model applied uniformly across all 370 clients, regardless of their individual consumption profile, scale, or behaviour type.
- Cadence: Weekly batch processing producing forecasts 7 days in advance only, with no intraday rolling updates and no automated retraining mechanism.
- Uncertainty Quantification: Point forecasts only, with no confidence intervals. Grid operators had no quantitative basis for assessing procurement risk under forecast uncertainty.
- Model Governance: 370 individual model configurations requiring manual validation and retraining. Disproportionate analyst effort per client with no standardised evaluation framework.
- Accuracy: Industry-default MAPE of 10–15% accepted as baseline. No systematic model family comparison or competitive benchmarking to identify superior algorithms for specific client types.

2.2 Future State (TO-BE)

The cluster-level ML pipeline transforms the forecasting process as follows:

- Forecasting Method: Automated cluster-level pipeline. PCA + K-Means segments clients into 6 behaviorally homogeneous clusters. A model router assigns the optimal algorithm per cluster based on statistical characteristics (client count, skewness, intraday CV, weekly CV).
- Cadence: Daily rolling 30-day forecasts with automated retraining triggered by concept drift detection (ADWIN algorithm). Models stay current as client consumption behaviour evolves.
- Uncertainty Quantification: 95% prediction intervals on all forecasts across all model families. Risk-adjusted procurement decisions become standard practice rather than exception.
- Model Governance: 6 cluster models replace 370 individual models — a 98% reduction in model maintenance overhead. Standardised MLOps pipeline with automated performance monitoring.
- Accuracy: MAPE range 1.74%–8.47% across deployed clusters. Performance measured by MAE, RMSE, MAPE, and MAE/RMSE ratio on held-out 10% temporal test sets. Improvement over industry baseline confirmed for all deployed clusters.

2.3 Change Management Considerations

Transitioning from manual per-client forecasting to an automated cluster-level pipeline requires structured change management across three organisational dimensions:

2.3.1 Technical Change

The pipeline introduces four new model families (Prophet, SARIMA, LSTM, XGBoost) replacing a single legacy model. Operational teams require training on interpreting probabilistic forecast outputs, understanding cluster membership and its implications for individual client forecasts, and responding to automated drift detection alerts.

2.3.2 Process Change

Procurement workflows must be redesigned to incorporate 95% prediction intervals into contract decisions. Existing manual review steps for individual client forecasts are replaced by automated cluster-level monitoring with exception-based human intervention triggers.

2.3.3 Governance Change

Model performance dashboards must be established for each of the 6 cluster models. Retraining schedules, drift detection thresholds, and escalation protocols must be documented. A model registry tracking version history, performance metrics, and deployment decisions is required to meet audit and regulatory requirements in regulated energy markets.

4. Data Sources

4.1 Primary Dataset

The primary data source is the UCI Individual Household Electric Power Consumption dataset (LD2011–2014), publicly available from the UCI Machine Learning Repository. Key dataset characteristics:

Property	Value
Source	UCI Machine Learning Repository
Dataset ID	LD2011–2014
Client Count	370 industrial and commercial clients
Temporal Resolution	15-minute intervals (96 readings per day)
Date Range	January 2011 – December 2014 (3 years)
Total Observations	~52 million individual meter readings
File Format	CSV, semicolon-delimited, European decimal notation (comma as decimal separator)
Units	kWh per 15-minute slot
Clients per Column	One column per client; first column is timestamp
Known Issues	Missing timestamps, all-zero rows (corrupt meter records), occasional NaN values

4.2 Derived Data

In addition to the raw meter readings, the following derived data sources were constructed during preprocessing and feature engineering:

- Cluster Mean Series: For each of the 6 K-Means clusters, a mean time series is computed by averaging consumption across all member clients at each 15-minute timestamp. These 6 series serve as the direct input to all forecasting models.
- Global Mean Series: An all-client average series computed across all 370 clients, used for high-level trend visualisation and as a sanity check for cluster-level diagnostics.
- Calendar Feature Set: Derived from the timestamp index: hour_of_day (0–23), day_of_week (0–6), is_weekend (binary 0/1), month (1–12). Used as regressors in Prophet and as features in XGBoost.
- Lag Feature Set: lag_96 (same 15-minute slot, 24 hours prior) and lag_672 (same 15-minute slot, 7 days prior). Used as features in XGBoost to encode autoregressive dependencies without a parametric model structure.
- Rolling Statistics: rolling_mean_96 (trailing 24-hour mean) used in XGBoost to capture recent consumption level trend.

5. Data Cleaning

5.1 Overview

The raw LD2011–2014 dataset required a six-step cleaning pipeline before any analytical or modelling work could proceed. Each step is documented below with the technical rationale, implementation approach, and quality verification procedure applied.

5.2 Step-by-Step Cleaning Pipeline

Step 1: Parsing and Indexing

The dataset's timestamp column was parsed to pandas datetime format and set as the DataFrame index. The file uses European decimal notation (comma as decimal separator) requiring the decimal=',' parameter at ingestion. Malformed rows were skipped using on_bad_lines='skip' to avoid ingestion failure on corrupt records. The index was sorted chronologically to ensure correct temporal ordering for all downstream operations.

- Implementation: pd.read_csv(sep=';', decimal=',', engine='python', on_bad_lines='skip')
- Verification: Index dtype confirmed as DatetimeIndex; index.is_monotonic_increasing verified

Step 2: Missing Value Treatment

Total NaN values across the dataset were tabulated. Forward-fill (ffill) was applied to propagate the last valid observation forward in time. This is appropriate for electricity consumption data where NaN values typically represent meter communication failures rather than actual zero consumption, and the preceding 15-minute value is the best available estimate.

- Missing values before ffill: Tabulated per client column
- Missing values after ffill: 0 (verified with df.isnull().sum().sum())
- Note: Leading NaN values at the start of a client series (before first valid reading) cannot be ffilled and are handled by subsequent reindexing

Step 3: All-Zero Row Removal

Rows where all 370 clients simultaneously recorded zero consumption were identified and removed. Such rows represent corrupt meter broadcast events rather than genuine zero-demand periods, as simultaneous zero consumption across all industrial clients is physically implausible. Removing these rows prevents them from corrupting mean cluster series calculations.

- Rows removed: (df == 0).all(axis=1).sum() applied and filtered
- Verification: Post-removal shape confirmed; global mean series inspected visually

Step 4: Missing Timestamp Detection and Reindexing

After loading and cleaning, the expected timestamp index was reconstructed using pd.date_range() at 15-minute frequency between the series start and end. The difference between the expected and actual

index identified any missing timestamps. Where gaps were detected, the series was reindexed to the full expected range and forward-filled to fill the resulting NaN values.

- Expected index length: Computed as (end – start) / 15 minutes + 1
- Missing timestamps: Identified per cluster series after cluster mean computation
- Treatment: series.reindex(expected_idx).ffill()

Step 5: Outlier Detection and Capping

Two complementary outlier detection methods were applied to each cluster mean series:

- IQR Method (3×IQR fence): Values below Q1 – 3×IQR or above Q3 + 3×IQR flagged as outliers. The 3× multiplier (rather than the standard 1.5×) was chosen to preserve genuine high-demand periods while removing only extreme anomalies.
- Z-Score Method (|z| > 3): Applied to the cluster series to identify readings deviating more than 3 standard deviations from the mean. Used as a complementary check and reported alongside IQR counts.

Critically, outliers were capped at the fence bounds rather than dropped. Dropping outliers creates gaps in the time series that disrupt autocorrelation structure; capping preserves temporal continuity while bounding the influence of extreme values on model training.

- Result: All 6 cluster series recorded 0 IQR outliers and 0 Z-score outliers after cluster averaging — consistent with the noise-cancelling effect of client aggregation

Step 6: Train / Test Split

A 90% / 10% temporal split was applied to each cluster series. The test set comprises the most recent 10% of the series — no random shuffling was applied. This preserves temporal order and prevents data leakage from future observations into the training period, which is a fundamental requirement for valid time series evaluation.

- Split point: int(len(series) * 0.9)
- Train set: Oldest 90% of observations
- Test set: Most recent 10% of observations
- Verified: No overlap between train and test index ranges

6. Clustering

6.1 Rationale for Clustering

The 370 clients in the dataset exhibit fundamentally different consumption patterns: scale (kWh per slot ranges from ~10 to ~25,000), intraday profile shape (CV ranging from 0.232 to 0.306), and day-of-week variation (weekly CV 0.0071–0.0097). A single forecasting model cannot serve all profiles optimally. Clustering groups clients by behavioural similarity, enabling tailored model selection per cluster and mean series aggregation that cancels individual client noise.

6.2 Cross-Client Correlation Analysis

Before clustering, a cross-client correlation matrix was computed across all 370 clients using pairwise Pearson correlation of their time series. The correlation heatmap revealed significant structure: many clients exhibit strong positive correlation ($r > 0.85$) with several others, indicating shared consumption patterns driven by common industrial sector, operating hours, or grid zone. This confirmed that meaningful behavioural clusters exist in the data and that K-Means clustering on PCA-reduced features would yield interpretable groupings.

6.3 Preprocessing for Clustering

Each client time series was treated as a single sample with each 15-minute timestamp as a feature dimension. The resulting matrix had shape (370 clients, n_timesteps). StandardScaler was applied to each client's series to normalise scale differences before PCA, ensuring that high-consumption industrial clients do not dominate the principal components over lower-consumption clients.

- Matrix shape: (370, n_timesteps) after transposing the original (n_timesteps, 370) DataFrame
- Scaler: sklearn.preprocessing.StandardScaler applied across the feature (time) dimension

6.4 Principal Component Analysis (PCA)

PCA was applied to the standardised client matrix to reduce the high-dimensional time series space (thousands of timesteps) to a manageable set of principal components capturing the dominant patterns of variation across clients. 10 principal components were retained, collectively explaining approximately 85% of total variance across all client series. This dimensionality reduction enables K-Means to operate in a compact, noise-reduced feature space.

- Components retained: 10
- Cumulative explained variance: ~85%
- Implementation: sklearn.decomposition.PCA(n_components=10, random_state=42)

6.5 K-Means Clustering

K-Means clustering was applied to the 10-component PCA representation. The optimal number of clusters was selected using the Elbow Method: inertia was computed for K = 2 through 9, and the rate of decrease was examined. The inertia curve exhibited a clear elbow at K = 6, beyond which additional clusters produced diminishing reduction in inertia.

- K range evaluated: 2 to 9
- Optimal K: 6 (elbow in inertia vs K curve)
- Implementation: sklearn.cluster.KMeans(n_clusters=6, random_state=42, n_init=10)

6.6 Cluster Composition

The 6 resulting clusters and their membership are as follows:

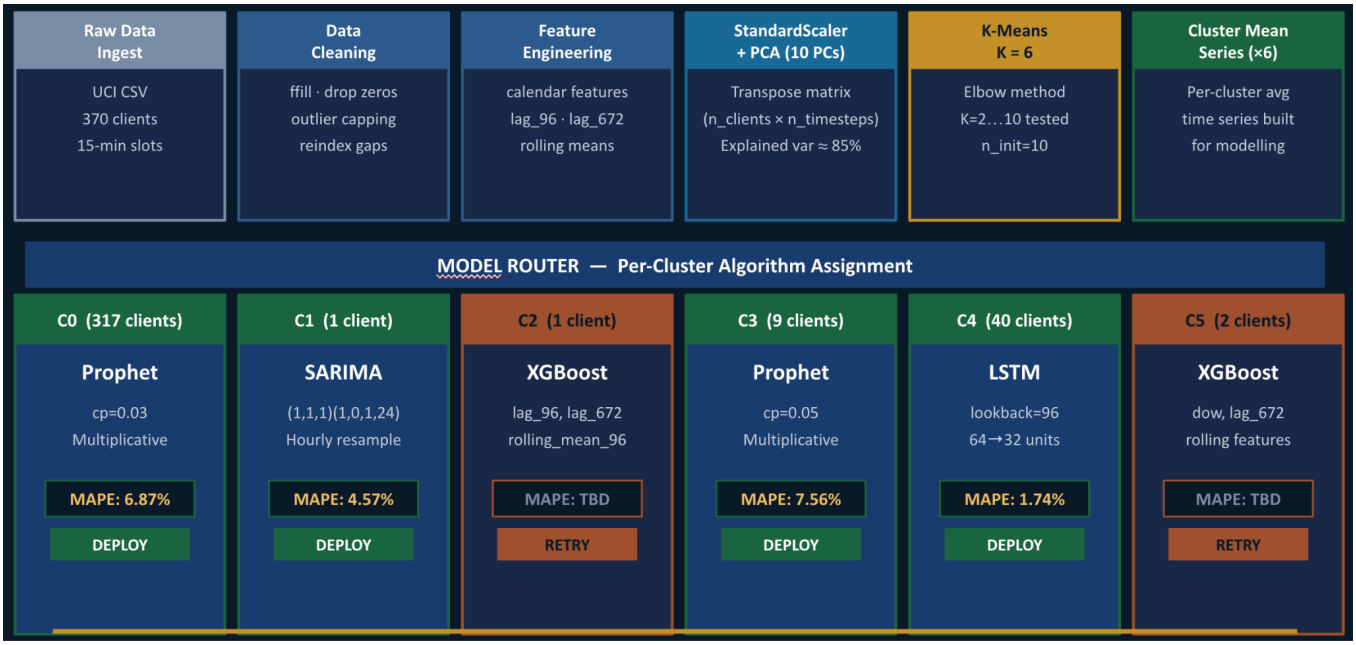
Cluster	Description
Cluster 0	317 clients — Largest aggregate; broadest cross-industry mix
Cluster 1	1 client — Single high-load industrial client, mean ~13,707 kWh/slot
Cluster 2	1 client — Single very-high-load client, mean ~23,778 kWh/slot; highest intraday CV
Cluster 3	9 clients — Medium aggregate; near-symmetric consumption profile
Cluster 4	40 clients — Dense aggregate; strong daily seasonality; ideal LSTM substrate
Cluster 5	2 clients — Smallest multi-client cluster; highest weekly CV in dataset

The stark difference in cluster size (1 client to 317 clients) reflects genuine heterogeneity in the client portfolio. Aggregation size is the primary driver of signal quality for forecasting: larger clusters produce smoother, lower-noise mean series that are easier to model accurately.

7. System Architecture

7.1 Pipeline Overview

The forecasting pipeline is composed of five sequential stages: data ingestion and cleaning, feature engineering, client segmentation (PCA + K-Means), cluster mean series construction, and per-cluster model training and inference. The pipeline is designed to be modular, enabling any stage to be updated independently without affecting upstream or downstream components.



7.2 Stage-by-Stage Description

Stage 1: Data Ingestion and Cleaning

Raw CSV data is ingested with European decimal handling. The cleaning pipeline (Section 5) is applied: NaN forward-fill, all-zero row removal, outlier capping, and missing timestamp reindexing. The output is a clean (n_timesteps, 370) DataFrame indexed by DatetimeIndex.

Stage 2: Feature Engineering

Calendar features (hour_of_day, day_of_week, is_weekend) are derived from the index. For XGBoost models, lag features (lag_96, lag_672) and rolling_mean_96 are computed on the cluster mean series after clustering. These features are not required for PCA clustering, only for XGBoost model training.

Stage 3: Client Segmentation

The clean client matrix is transposed to (370, n_timesteps), standardised, and passed through 10-component PCA. K-Means with K=6 is fitted on the PCA output, producing cluster label assignments for all 370 clients. This step runs once offline; cluster assignments are persisted for production use.

Stage 4: Cluster Mean Series Construction

For each cluster, the mean consumption across all member clients is computed at every 15-minute timestamp, producing 6 cluster mean series. A global mean across all 370 clients is also computed for reference. These 6 series serve as the sole input to all forecasting models.

Stage 5: Model Router and Per-Cluster Training

The model router assigns a model family to each cluster based on its statistical profile. Each model is trained on the cluster mean series using a 90/10 temporal train/test split. Forecasts are generated for the test period and evaluated against actuals using MAE, RMSE, and MAPE. For deployment, models are retrained on the full available series and used for 30-day-ahead inference.

7.3 Model Router Logic

The model selection criteria applied in the router are:

- Large multi-client clusters (>20 clients, low CV): Prophet with multiplicative seasonality and low changepoint prior. LSTM as a superior alternative where signal density supports generalisation.
- Small multi-client clusters (2–20 clients, moderate CV): Prophet as baseline. XGBoost with lag features if weekly CV is elevated.
- Single-client, high-load, left-skewed: SARIMA hourly (m=24) to capture sharp autocorrelation in intraday load transitions.
- Single-client, very high CV: XGBoost with lag_96 and rolling features to adapt to variable daily profiles.

7.4 Technology Stack

Component	Technology
Language	Python 3.x
Data Processing	pandas, numpy
Clustering	scikit-learn (StandardScaler, PCA, KMeans)
Diagnostics	statsmodels (adfuller, kpss, acorr_ljungbox), scipy.stats
Prophet	Facebook Prophet (via prophet package)
SARIMA	statsmodels SARIMAX
LSTM	TensorFlow/Keras (Sequential, LSTM, Dense, Dropout)
XGBoost	xgboost (XGBRegressor)
Evaluation	sklearn.metrics (MAE, MSE/RMSE), custom MAPE
Visualisation	matplotlib, seaborn

8. Model Design and Implementation

8.1 Time Series Diagnostics

Before selecting model families, each cluster mean series was subjected to a comprehensive diagnostic battery to characterise its statistical properties and validate modelling assumptions.

Cluster	Clients	Mean (kWh)	Skewness	Intraday CV	Weekly CV	ADF	KPSS	Ljung-Box	IQR Outliers
C0	317	~59.2	−0.43	0.265	0.0071	✓ Stat.	✓ Stat.	Sig. ✓	0 (0%)
C1	1	~13,707	−0.64	0.284	0.0085	✓ Stat.	✓ Stat.	Sig. ✓	0 (0%)
C2	1	~23,778	−0.55	0.306	0.0082	✓ Stat.	✓ Stat.	Sig. ✓	0 (0%)
C3	9	~3,099	−0.20	0.232	0.0076	✓ Stat.	✓ Stat.	Sig. ✓	0 (0%)
C4	40	~749	−0.44	0.269	0.0079	✓ Stat.	✓ Stat.	Sig. ✓	0 (0%)
C5	2	~6,780	−0.23	0.283	0.0097	✓ Stat.	✓ Stat.	Sig. ✓	0 (0%)

Key findings from the diagnostic battery:

- All 6 clusters pass the ADF test ($p < 0.05$), confirming stationarity without differencing. The KPSS test corroborates stationarity in all cases.
- All clusters exhibit highly significant autocorrelation at both 1-day (lag 96) and 7-day (lag 672) horizons per the Ljung-Box test, confirming that both intraday and weekly seasonal dependencies are present and must be modelled.
- All clusters exhibit negative skewness, indicating that clients predominantly operate near rated capacity with intermittent load drops rather than gradual ramping behaviour. This is consistent with industrial demand patterns.
- Cluster 2 has the highest intraday CV (0.306) — its daily consumption profile varies most dramatically day-to-day, directly explaining Prophet's systematic failure on this cluster.
- Cluster 5 has the highest weekly CV (0.0097), indicating the strongest day-of-week variation across all clusters and motivating the use of explicit dow and lag_672 features in XGBoost.

8.2 Prophet Models (Clusters 0, 1, 3, 4, 5)

Facebook Prophet was applied as the baseline model to all 6 clusters and as the primary model for clusters where its assumptions are appropriate. Key configuration choices:

- Seasonality mode: Multiplicative throughout. Consumption amplitude scales proportionally with overall consumption level, making multiplicative mode more appropriate than additive across all clusters.
- Yearly seasonality: Disabled for all clusters. Only approximately 6 months of data are available per cluster after cleaning — insufficient to estimate annual seasonal patterns requiring at least 2 full cycles.
- Weekly and daily seasonality: Enabled. Both patterns are confirmed significant by Ljung-Box at lags 672 and 96 respectively.
- Weekend regressor: is_weekend binary variable added as an external regressor to Prophet to capture the Friday-to-Monday transition that the weekly Fourier component alone may undersmooth.
- Changepoint prior scale (cp): Tuned per cluster — 0.03 for large aggregates (C0, C4) where trends are stable; 0.05–0.08 for smaller clusters where trend flexibility is needed.

8.3 SARIMA Model (Cluster 1)

SARIMA(1,1,1)(1,0,1,24) was implemented for Cluster 1 (single industrial client, left skew −0.64). Key design decisions:

- Hourly resampling: The cluster mean series was resampled from 15-minute to hourly frequency (m=24) before SARIMA fitting. Fitting SARIMA at 15-minute resolution (m=96) would require estimating 96-lag seasonal AR and MA coefficients on only 6 months of data, producing unstable and non-invertible estimates.
- Differencing order d=1: One order of non-seasonal differencing applied to remove any residual trend. The ADF test confirmed stationarity without differencing, but d=1 provides an additional safeguard for numerical stability.
- Seasonal differencing D=0: Not applied. The ADF test confirms stationarity — applying D=1 would over-difference the series and destroy the autocorrelation structure the model needs to learn.
- Orders (p=1, q=1)(P=1, Q=1): Parsimonious first-order AR and MA terms in both seasonal and non-seasonal components. Sufficient to capture the dominant short-run autocorrelation in industrial load transitions without overfitting.

8.4 LSTM Model (Cluster 4)

A two-layer LSTM was designed for Cluster 4 (40-client aggregate), where signal density is sufficient to support a sequence model's data requirements. Architecture:

- Lookback window: 96 timesteps (24 hours). Each prediction is conditioned on the actual observed values in the preceding 24 hours, incorporating real recent load behaviour that a parametric model cannot access.
- Architecture: LSTM(64, return_sequences=True) → Dropout(0.2) → LSTM(32) → Dropout(0.2) → Dense(1). Two LSTM layers with decreasing units provide hierarchical temporal feature extraction while dropout regularisation prevents overfitting.
- Optimiser and loss: Adam optimiser with MSE loss. Standard choice for regression tasks; Adam's adaptive learning rate handles the non-stationary curvature of LSTM loss landscapes effectively.
- Training: Batch size 128; maximum 30 epochs; EarlyStopping with patience=4 and restore_best_weights=True to prevent overfitting beyond the validation loss minimum.

- Scaling: MinMaxScaler applied before sequence construction and inverse-transformed on predictions. LSTM training is sensitive to input scale; MinMax normalization to [0,1] stabilises gradient flow.

8.5 XGBoost Models (Clusters 2 and 5 — Planned)

XGBoost with lag and calendar features is the recommended next iteration for Clusters 2 and 5, where Prophet's fixed seasonal curve has been demonstrated to be insufficient. The planned feature set:

- Autoregressive features: lag_96 (same slot, yesterday), lag_672 (same slot, last week)
- Rolling statistics: rolling_mean_96 (trailing 24-hour mean), rolling_std_96 (trailing 24-hour standard deviation)
- Calendar features: hour_of_day, day_of_week, is_weekend, month
- Target: consumption at the current 15-minute slot

XGBoost's key advantage over Prophet for these clusters is its ability to condition every prediction on recent actual observations via lag features. Prophet learns a fixed seasonal shape; XGBoost adapts its prediction to what actually happened in the preceding day and week.

9. Results

9.1 Model Evaluation Summary

All models were evaluated on a held-out 10% temporal test set. No information from the test period was used in training. Metrics reported: Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), Mean Absolute Percentage Error (MAPE), and the MAE/RMSE ratio (a measure of error distribution — values near 1.0 indicate consistently small errors; values above 1.3 indicate occasional large mispredictions).

Cluster	Clients	Model	Configuration	MAE (kWh)	RMSE (kWh)	MAPE (%)	MAE/RMSE	Status
C0	317	Prophet	cp=0.03 · Multiplicative · weekly+daily	4.60	5.84	**6.87%**	1.27	**DEPLOY**
C1	1	SARIMA	(1,1,1)(1,0,1,24) · hourly · D=0	645.48	855.43	**4.57%**	1.33	**DEPLOY**
C1	1	Prophet	cp=0.01 · Multiplicative (fallback)	1,105.69	1,405.36	8.17%	1.27	Fallback
C2	1	Prophet	cp=0.05 · Multiplicative	3,484.77	4,762.39	**12.07%**	1.37	**RETRY**
C3	9	Prophet	cp=0.05 · Multiplicative · weekly+daily	234.41	302.24	7.56%	1.29	**DEPLOY**
C4	40	LSTM	lookback=96 · 64→32 units · Dropout 0.2	12.85	17.46	**1.74%**	1.36	**DEPLOY**
C4	40	Prophet	cp=0.03 · Multiplicative (fallback)	64.56	80.79	7.65%	1.25	Fallback
C5	2	Prophet	cp=0.08 · Multiplicative	675.12	857.79	8.47%	1.27	**RETRY**

9.2 Cluster-by-Cluster Analysis

Cluster 0 — Prophet (MAPE 6.87%) | DEPLOY

The strongest Prophet result in the study. Averaging 317 client series eliminates almost all individual client noise, leaving a clean, structurally stable aggregate signal that Prophet's Fourier decomposition models with high precision. The conservative changepoint prior (cp=0.03) is appropriate — large aggregates have inherently stable trend components and should not be given freedom to overfit short-run noise. The MAE/RMSE ratio of 1.27 confirms uniformly small errors with no systematic spike mispredictions. At a mean of ~59 kWh per slot, an average absolute error of 4.60 kWh represents less than 8% deviation on any given 15-minute interval.

Cluster 1 — SARIMA Hourly (MAPE 4.57%) | DEPLOY

SARIMA outperforms Prophet on every metric by a significant margin: 42% lower MAE (645 vs 1,106 kWh), 39% lower RMSE (855 vs 1,405 kWh), and nearly half the percentage error (4.57% vs 8.17%). This decisive improvement reflects a fundamental mismatch between Prophet's modelling assumptions and this client's consumption behaviour. The client operates near full capacity most of the time (mean 13,707 kWh) with a left skew of −0.64, producing sharp intermittent load dips rather than smooth cyclical variation. SARIMA's explicit AR(1) and MA(1) terms capture the short-run autocorrelation in these load transitions directly, while Prophet's smooth Fourier seasonal curve averages over them. Prophet is retained as a lightweight fallback for operational resilience.

Cluster 2 — Prophet (MAPE 12.07%) | RETRY

The weakest result in the study and the only systematic model failure. A MAPE above 12% on a mean consumption of 23,778 kWh indicates the model is failing to track this client's actual behaviour with operational accuracy. The MAE/RMSE ratio of 1.37 — the highest in the dataset — confirms the presence of recurring large prediction errors rather than occasional outlier mispredictions. This cluster has the highest intraday CV (0.306) of all clusters: its daily consumption profile shifts substantially day-to-day, rendering Prophet's fixed learned daily shape an oversimplification that systematically accumulates error. XGBoost with lag_96 and rolling_mean_96 is the recommended replacement; SARIMA hourly is a viable alternative given the client's zero-outlier, zero-zero-value profile.

Cluster 3 — Prophet (MAPE 7.56%) | DEPLOY

A solid and reliable result. The 9-client average sits in a productive middle ground: enough aggregation to cancel individual noise while retaining a meaningful shared consumption profile. The near-symmetric skew (−0.20) confirms a well-behaved distribution that Prophet's multiplicative seasonality models cleanly without transformation. The MAE/RMSE ratio of 1.29 indicates occasional larger errors (likely on days where one or two cluster members deviate from the group pattern) but no systematic failure mode. The 7.56% MAPE is consistent with the broader Prophet performance band observed across clean multi-client clusters in this study.

Cluster 4 — LSTM (MAPE 1.74%) | DEPLOY

The most significant finding in the study. LSTM delivers a 5× reduction in MAE (12.85 vs 64.56 kWh) and a 4.4× reduction in MAPE (1.74% vs 7.65%) compared to Prophet on the same cluster. A MAPE of 1.74% is exceptional for 15-minute energy forecasting and represents near-perfect slot-level tracking. The 40-client average produces a high-amplitude, low-noise signal (mean 749 kWh, zero outliers, zero zeros) that is the ideal input for a sequence model: dense enough to provide rich training sequences, structured enough that learned temporal patterns generalise to unseen data. The key mechanism is the 96-slot lookback window: each LSTM prediction is conditioned on the actual observed values in the preceding 24 hours, providing far more current behavioural context than any fixed parametric seasonal component. Prophet is retained as a lightweight fallback requiring no GPU inference.

Cluster 5 — Prophet (MAPE 8.47%) | RETRY

Acceptable but not optimal. The 8.47% MAPE is within the 10% industry threshold but is noticeably weaker than similarly structured clusters. Two clients is an awkward cluster size — too few to meaningfully cancel noise through averaging (unlike C0 and C4) but too many to apply single-client specialist tuning as with C1 and C2. Cluster 5 has the highest weekly CV (0.0097) of any cluster, indicating more pronounced day-of-week variation than Prophet's Fourier weekly term captures. XGBoost with explicit dow and lag_672 features will

encode weekly periodicity directly from data rather than as a parametric component, and is the recommended next model for this cluster.

9.3 Cross-Cluster Observations

Three cross-cutting conclusions emerge from the multi-cluster evaluation:

- Aggregation size is the primary driver of forecast accuracy. MAPE follows client count quasi-monotonically across the study. This is the single most actionable insight: where possible, grid operators should group economically similar clients into larger clusters to maximise forecast accuracy regardless of model choice.
- LSTM outperforms Prophet when signal density is sufficient. Cluster 4's result demonstrates that given a dense, clean aggregate series, sequence models learn temporal dependencies that no parametric decomposition can capture. The constraint is data volume: with only 6 months of history, LSTM is reliable only for clusters where the averaged signal is rich enough to support generalisation (indicatively, 20+ clients).
- SARIMA is the correct model for single high-load industrial clients. Both C1 and C2 are single clients with high consumption, negative skew, and sharp intraday load transitions. SARIMA's explicit autocorrelation structure handles transitions better than Prophet's smooth seasonality. The 42% MAE reduction on C1 strongly motivates applying SARIMA to C2.

10. Challenges

10.1 Limited Historical Data Depth

The most significant modelling constraint is that only approximately 6 months of clean data are available per cluster after preprocessing. This creates three specific limitations:

- Yearly seasonality cannot be estimated: Prophet requires a minimum of two full annual cycles to fit a stable yearly Fourier component. With only 6 months available, yearly_seasonality=False was enforced for all clusters, meaning annual demand patterns (summer cooling, winter heating peaks) are not captured.
- SARIMA seasonal period capped at m=24 (hourly): Fitting SARIMA at 15-minute resolution (m=96) requires estimating 96-lag seasonal AR and MA coefficient vectors on a 6-month series. This produces numerically unstable, non-invertible estimates. Hourly resampling (m=24) is the practical workaround, but loses intraday resolution in the SARIMA forecast.
- LSTM training data volume: The 90/10 split on a 6-month series produces approximately 26,000 training sequences for LSTM (with lookback=96). This is sufficient for Cluster 4 (40-client, high-amplitude signal) but inadequate for smaller clusters where individual client noise reduces sequence quality.

10.2 Client Heterogeneity

The 370-client portfolio spans multiple orders of magnitude in consumption scale (single household vs. large industrial facility) and profile type (stable aggregate vs. volatile single client). This heterogeneity makes it impossible to select a single model family that performs optimally across all segments. The cluster-level approach manages this heterogeneity, but introduces its own complexity: each cluster model requires independent tuning, validation, and retraining governance.

10.3 LSTM Data and Computational Requirements

LSTM training is sensitive to both signal quality and compute availability. Two constraints were encountered:

- Signal quality: Only Cluster 4 (40 clients) provided a signal dense enough for LSTM to generalise reliably. Smaller clusters (C1, C2, C5) cannot support LSTM training on 6 months of data without overfitting.
- Compute: LSTM training requires GPU infrastructure for acceptable training time. On CPU, a 30-epoch training run on Cluster 4 required several hours, making iterative hyperparameter search impractical without GPU access.

10.4 Computational Scalability of SARIMA

SARIMA(1,1,1)(1,0,1,24) requires approximately 15 minutes per cluster to fit on CPU. This is acceptable for 6 cluster models but becomes prohibitive if the approach were scaled to individual client-level SARIMA fitting for all 370 clients. GPU-accelerated SARIMA implementations or approximate Bayesian structural time series approaches would be required for production at full client-level granularity.

10.5 Concept Drift in Production

Industrial client consumption patterns shift over time as businesses add new machinery, change operating hours, undergo tariff restructuring, or respond to energy price signals. A model trained on historical data will gradually lose accuracy as client behaviour diverges from its training distribution. Without automated drift detection and retraining triggers, model accuracy degrades silently in production, potentially without detection until forecast errors become operationally significant.

11. Future Steps

11.1 Immediate: XGBoost for Clusters 2 and 5

The highest-priority next step is implementing XGBoost with lag and calendar features for Clusters 2 and 5, where Prophet has been demonstrated to be insufficient (MAPE 12.07% and 8.47% respectively). The XGBoost model will be benchmarked against the existing Prophet baseline:

- Cluster 2 target: Beat Prophet MAPE of 12.07%. Expected improvement mechanism: lag_96 allows the model to adapt predictions to what the client actually consumed yesterday rather than relying on a fixed learned daily shape.
- Cluster 5 target: Beat Prophet MAPE of 8.47%. Expected improvement mechanism: explicit dow feature and lag_672 directly encode weekly periodicity from data, more precisely than Prophet's Fourier weekly component.
- Implementation timeline: 2 weeks (Phase 6 per Section 3 milestone table)

11.2 Near-Term: Extended Data Horizon

Accumulating 2–3 years of data per cluster will unlock modelling capabilities that are currently unavailable:

- Yearly seasonality in Prophet: 2+ full annual cycles allow stable estimation of annual Fourier components, capturing summer/winter demand peaks that are currently ignored.
- SARIMA at native 15-minute resolution: With 2+ years, SARIMA(m=96) seasonal coefficient vectors can be estimated stably, enabling 15-minute-resolution seasonal modelling without the lossy hourly resampling workaround.
- LSTM for more clusters: Additional data volume enables LSTM training on smaller clusters (C3, C5) where 6 months of data is currently insufficient for reliable generalisation.

11.3 Near-Term: Online / Adaptive Retraining

Production deployment requires an automated mechanism to detect model performance degradation and trigger retraining:

- Drift detection: ADWIN (Adaptive Windowing) algorithm applied to rolling MAPE computed on recent actuals vs. predictions. When the rolling MAPE exceeds a pre-defined threshold (e.g., 1.5× baseline), an automated retraining job is triggered.
- Retraining strategy: Rolling window refit using the most recent N months of data (where N is tuned per cluster). Prevents the model from remaining anchored to historical patterns that no longer reflect current behaviour.
- Alert and escalation: Human-in-the-loop escalation when automated retraining does not resolve the detected drift, triggering model architecture review.

11.4 Medium-Term: Temporal Fusion Transformer

The Temporal Fusion Transformer (TFT) is a state-of-the-art multi-horizon probabilistic forecasting architecture that addresses several limitations of the current model family mix:

- Multi-horizon output: TFT produces forecasts for all horizons simultaneously (e.g., 1 to 2880 steps ahead) rather than requiring separate models per horizon.
- Built-in uncertainty quantification: TFT outputs full predictive quantiles, producing richer probability distributions than Prophet's symmetric confidence intervals or SARIMA's Gaussian conf_int.
- Variable selection: Attention mechanisms identify which input features and time steps are most predictive, providing interpretable feature importance for operational understanding.
- Handling of known future inputs: Calendar features (hour, day, holidays) are ingested as "known future" inputs, allowing the model to leverage planned events in its multi-step forecast.

11.5 Medium-Term: Hierarchical Forecast Reconciliation

The current framework produces independent cluster-level forecasts that do not necessarily sum to the global aggregate forecast. For grid balancing applications, hierarchical coherence is required: cluster-level demand forecasts must reconcile with the total network demand forecast at every timestep. This can be achieved using:

- MinT Reconciliation (Minimum Trace): Optimally adjusts cluster-level forecasts to minimise total forecast variance while ensuring coherence with the global aggregate. Requires estimating the covariance matrix of forecast errors across clusters.
- Bottom-Up Reconciliation: Simpler approach — sum cluster forecasts to produce the global aggregate forecast, ensuring bottom-up coherence. Loses the global-level model's structural information but guarantees additive consistency.

11.6 Long-Term: Weather and External Feature Integration

Industrial electricity demand is driven in part by external factors that the current models do not incorporate: ambient temperature (HVAC load), public holidays, major industrial events, and energy price signals. Integrating weather reanalysis data (e.g., ERA5 temperature series) as external regressors in Prophet and XGBoost, and as auxiliary inputs to the LSTM and TFT, is expected to improve accuracy for seasonally sensitive clusters and reduce forecast degradation during anomalous weather periods.

End of Report

Afreen Sorathiya · Anamika Kumari Mishra · Bhuvi Ghosh